

RESEARCH ARTICLE

Bridging Chess Engines and Language Models: A Novel Framework for Generating Human-Like Chess Commentary

HITESH SHIVKUMAR¹, PRANAV BHATLAPENUMARTHI¹, SABHYA GUPTA¹,
TANISHKA SRIVASTAVA¹, SWATHI JAMJALA NARAYANAN¹, (Senior Member, IEEE),
AND BOOMINATHAN PERUMAL¹, (Senior Member, IEEE)

School of Computer Science and Engineering, Vellore Institute of Technology, Vellore 632014, India

Corresponding author: Swathi Jamjala Narayanan (jnsuawathi@vit.ac.in)

ABSTRACT This paper proposes a reinforcement learning (RL)-based framework that automates chess commentary generation using real-time board analysis. It combines chess engine evaluation along with natural language generation to produce strategically grounded commentary which is also linguistically fluent. The framework consists of three major stages: 1) a board-state evaluation layer that makes use of Stockfish engine outputs to analyse positions, 2) a structured annotation layer based on Proximal Policy Optimisation (PPO) that generates quality, phase, and tactical commentary tokens, and 3) a language generation layer utilizing leading Large Language Models (LLMs) such as GPT-4o, Gemini 2.5 Pro, and Gemma 4 to synthesize natural language commentary. A seven-metric automated evaluation framework was developed that incorporates GPT-2 perplexity, DeBERTa-v3 Natural Language Inference (NLI), Sentence-BERT similarity, MTLD and Self-BLEU to evaluate the generated commentary. Experiments on championship-level games show that the framework balances factual consistency, strategic depth, and natural language fluency quite effectively, while revealing measurable trade-offs between various LLMs. The proposed system demonstrates the potential of combining RL with generative AI for application purposes in live chess broadcasting, educational platforms, and intelligent game analysis systems.

INDEX TERMS Chess commentary generation, reinforcement learning, proximal policy optimization, intermediary model, special tokens, natural language generation.

I. INTRODUCTION

The domain of sports commentary has seen significant changes with the advancements of Large Language Models (LLMs). The traditional method of human commentators providing the viewers with move-by-move narration and analytical insights for an ongoing game has been revolutionised with the advent of AI-driven commentary and evaluation systems, which enhance and personalise viewer engagement. Remarkable capabilities have been demonstrated by LLMs such as Anthropic AI's Claude models, OpenAI's GPT models, Google's Gemini models and Meta's LLaMA models in understanding natural language prompts and generating context-aware and stylistically rich commentary. Chess commentary plays a significant role in improving the spectator's

experience, giving reasonable and strategic insights behind every move. Traditionally, knowledgeable hosts, along with elite players, deliver move analysis, predictions and engaging narratives about the ongoing game. This approach, though effective, demands significant coordination, preparation, and domain-specific expertise to give the viewers high-quality commentary which genuinely captures their attention. Even today, online chess tournaments and global events face significant hiccups in catering chess commentary to a diverse audience, as their understanding and perception of the game is highly varied.

The field of automated chess commentary generation serves as a great blend of Artificial Intelligence, Natural Language Processing (NLP) and Game Theory. In contrast to traditional chess engines that rely on calculating optimal moves and position evaluations, the generation of commentary requires technically correct and strategic insights,

The associate editor coordinating the review of this manuscript and approving it for publication was Xin Sun¹.

and at the same time, it needs to be engaging, capturing emotional and dramatic nuances that improve the viewer's experience. These requirements and experiences are not wholesome in the current chess analysis tools and existing language models, suggesting a valid space for improvement. Modern chess engines include Stockfish, AlphaZero, Leela Chess Zero (LCZero), and Komodo which compute and provide near-perfect evaluation of chess positions expressed as numeric scores indicative of the advantage or disadvantage for a particular player, but they miss out on human-like interactive explanations, insights and conversational engagement. Amidst the recent advancements, multimodal models like Gemma 4 and GPT-4 stood out in technical and conversational tasks. These models are capable of generating human-like natural language explanations, providing strategic reasoning behind moves, drawing comparisons to historical games and creating engaging commentary for both educational and entertainment purposes. These models help in combining chess intelligence with natural language fluency, making chess more accessible, fun, engaging and instructive for a wide range of audiences.

However, existing methodologies primarily focus on training AI systems extensively on game rules and historical data to produce the desired outcomes [1], [2]. Despite being innovative, this approach proves to be computationally expensive, resource-intensive, and time-consuming, requiring substantial training datasets and continuous fine-tuning to accommodate evolving game scenarios [3]. Current solutions lack an integrated framework capable of real-time board position analysis and commentary synthesis that balances strategic depth with audience engagement. This creates a gap in delivering accessible, dynamic, and high-quality commentary for a wide range of audiences. Therefore, there remains a need for an integrated framework capable of generating context-aware and strategically accurate chess commentary in real time.

To address these challenges, a multi-layered framework for real-time commentary generation is proposed in this paper, utilising Generative AI to integrate board position analysis with natural language synthesis. The key innovation lies in introducing a reinforcement learning-based intermediary model that analyses the current move based on the score generated by the chess engine and produces corresponding intermediary tokens that contain the necessary key points for generating human-like commentary. This framework aims to reduce computational overhead, enhance adaptability, and improve the quality of automated chess commentary while ensuring real-time responsiveness.

II. REVIEW OF LITERATURE

A. REVIEW OF METHODS ON CHESS PLAYING ENGINES

The study by Maharaj et al. [4] discusses how the evaluation of the decision-making capabilities of chess engines has advanced using AI. Chess endgame studies serve as a benchmark for AI performance, as they require deep calculation and strategic foresight. A critical test for evaluating the

performance of various AI algorithms in handling complex positions is Plaskett's puzzle. Stockfish and LCZero are two significant chess engines showing contrasting approaches to AI-based decision-making. Stockfish relies on alpha-beta pruning and deep iterative search, which makes it suitable for tactical and computationally intensive problems, whereas Leela Chess Zero (LCZero) utilises deep reinforcement learning techniques paired with Monte Carlo Tree Search (MCTS) that focuses on pattern recognition and probabilistic evaluations. LCZero's selective search mechanism has limitations, which are highlighted by the better performance of Stockfish while solving Plaskett's puzzle. LCZero fails to prioritise critical lines of play because of its heuristic-based learning process rather than brute-force search. A key discussion in the paper revolves around generalisation in AI. While Stockfish is finely trained for chess with extensive human-crafted heuristics, LCZero represents a more generalizable approach that could extend to other games. Baxter et al. [5] introduced TDLEAF, a modified version of the TD algorithm designed to work effectively alongside game-tree search. In game AI, TD-Gammon, a backgammon-playing neural network, became very successful, attaining expert human levels using self-play. However, for chess, applying TD-learning is quite challenging due to differences in game structure and search depth requirements. To overcome this issue, TDLEAF approach has been put into effect. Instead of updating game states directly, TDLEAF(λ) updates the leaf nodes of the principal variation (i.e., the best sequence of moves found by the search algorithm). This allows the evaluation function to improve within a deep search framework. Chess program "KnightCap" used TDLEAF to learn its evaluation function while playing on publicly available online chess servers (it benefited from online play against human opponents in contrast to self-play). The main success reported is that KnightCap, while not taking much time, improved from an intermediate level rating of 1650 to an extremely strong rating of 2150.

The work by Noever et al. [6] demonstrates how transformer-based models can be trained on Portable Game Notation (PGN), which accurately encodes entire chess games as text sequences. GPT-2 and other transformers can generate reasonable and strong chess moves based on game history after being trained to infer rules purely from data, without any manual intervention. Strong performance requires the transformer to have access to the entire game history, including complete board state and move sequence. Large training datasets improve the model's ability to track board state and generate strong moves, whereas significant drops in prediction accuracy occur due to approximate attention mechanisms or truncated histories. Models can sometimes violate chess rules while generating moves. Therefore, illegal move filtering is required (for example-Stockfish-based validation). The model's skill can in turn be evaluated with the help of training data segmentation into beginner and advanced games or comparing different model sizes (for example - 124M vs. 774M parameter GPT-2). The

study by Monroe et al. [7] compares the working algorithms of chess engines and transformer-based chess models. Chessformer, a transformer-based chess model, learns long-range dependencies better than convolution-based architectures like those used in AlphaZero. The encoded positional information in the attention mechanism accounts for the performance of transformers in chess. The paper compares unique strengths of Chessformer over traditional engines, including its understanding of trapped pieces (pieces with no escape squares), fortresses (closed positions where no progress can be made), positional imbalances that are difficult for traditional engines to evaluate. It excels in performance with significantly fewer computational resources than prior models. Transformer-based models are more interpretable than traditional neural network-based chess engines, with a focus on positions' defensive resources and critical threats, and key tactical motifs. Thus, it can be leveraged to generate human-like commentary by learning from annotated chess games rather than evaluating moves numerically.

B. REVIEW OF METHODS ON SPORTS COMMENTARY USING LLMs

The work done by Cook et al. [8] highlights the challenges faced in real-time commentary generation for football matches owing to the complexity of natural language, domain-specific knowledge, and the dynamic nature of matches which calls for advancements in NLP and LLMs. Earlier automated sports commentary lacked flexibility and adaptability, but now, transformer-based models have demonstrated superior text generation capabilities, enabling more coherent and contextually relevant outputs. Fine-tuning LLMs for sports commentary comes with difficulties like catastrophic forgetting, domain adaptation, and real-time processing limitations. Open-source models such as LLaMA allow efficient fine-tuning on consumer-grade hardware which help in combating the abovementioned problems. Building on previous research, the study introduces LLM-Commentator, a specialized model designed for real-time football commentary. It comes with three innovative fine-tuning approaches: Layered Model (LM), Mixed Immediately Model (MIM), and Mixed Sequentially Model (MSM) to enhance coherence, precision, and adaptability. The study conducted by Ge et al. [9] explored the significant advancements that have been made in multi-modal learning with the rise of Video Large Language Models (Video-LLMs), particularly in video analysis tasks, but the existing benchmarks fail in comprehensively evaluating fine-grained temporal visual capabilities. It relies on simple videos and limited task formats which don't assess a model's ability to generate detailed and context-aware textual outputs sufficiently. The paper presents SCBench, a novel evaluation framework designed specifically for sports video commentary generation, to address these limitations. It has two key contributions: (a) SCORES, a six-dimensional metric for commentary quality, and (b) CommentarySet, a dataset

of 5,775 annotated sports video clips across six sports. In contrast to traditional captioning datasets, CommentarySet includes professional commentary covering key events, tactics, emotions, and context. The paper highlights performance gaps in sports commentary generation by evaluating multiple Video-LLMs, including VILA, Video-LLaVA, and InternVL-Chat-2. Wang et al. [10] introduced a multimodal e-sports commentary generation framework that uses gameplay screenshots and structured JSON-based game data to generate commentary for specific gameplay moments in strategic multiplayer games. The study constructed a large-scale tri-modal dataset for the game "League of Legends" and evaluated multiple vision-language models using automatic metrics and human judgements focused on relevance and accuracy of generated commentary. The work showcases the importance of combining structured and visual modalities to improve contextual understanding and quality of the commentary in dynamic environments. It targets multimodal e-sports narration and vision-language integration, whereas the proposed framework emphasizes chess-specific strategic analysis. The study conducted by Kameko et al. [1] presents a machine-learning based approach which involves natural language comment generation on Shogi games (Japanese Chess). They deal with a system which provides textual, human-like commentary based upon the automatic analysis of a Shogi game record, which in turn helps non-experts understand the reason behind each move. The methodology is divided into two machine-learning steps a) Prediction of keywords and phrases that characterize the Shogi position. b) These words are then used to generate full comments using a discriminative log-linear language model trained on a dataset of human expert commentary. Central to their method is accurate mapping of textual expressions from expert comments to specific game positions, which is achieved using evaluation information from a Shogi program.

C. REVIEW ON LARGE LANGUAGE MODELS

The study by Mohammad et al. [11] presents how LLMs use NLP to enable human-computer interaction by facilitating tasks like machine translation, sentiment analysis, speech recognition, and summarization by employing techniques such as Named Entity Recognition (NER), Part-of-Speech (POS) tagging, and parsing. The paper explores key NLP components: (a) Natural Language Understanding (NLU) for text comprehension, and (b) Natural Language Generation (NLG) for text synthesis. NLP includes limitations/challenges like ambiguity, word boundary detection, and varied sentence structures by which language interpretation is affected. Mitigating these difficulties, deep learning-based NLP (particularly models like GPT) has significantly advanced speech synthesis and automatic speech recognition. Applications, limitations, and future scope of LLMs are detailed in this paper showcasing their potential to bridge human-machine communication gaps. The exploration by Almazrouei et al. [12] imparts insights on the Falcon

series, consisting of 7B, 40B, and 180B parameter causal decoder-only models trained on a high-quality, large-scale dataset primarily sourced from web data. Falcon-180B is the largest model trained on 3.5 trillion tokens which makes it the most extensive openly documented pretraining run to date, models like PaLM and Chinchilla and rivals such as LLaMA-2 and Inflection-1 are outperformed by it. Falcon series is scalable (efficient pre-training on 4,096 A100 GPUs), data efficient (RefinedWeb dataset - 5 trillion tokens), advanced model (including valid modifications for improved efficiency) and has open access - Falcon-7B, 40B, and 180B have an open license, along with a 600B-token extract of RefinedWeb. Challenges of large-scale training, including data sensitivity, inference cost management, and optimal pre-training strategies, are mentioned in the paper, accompanied by enhanced fine-tuning, improved long-context processing, and integration of more code data for future works. The research work by Aridoss et al. [13] frames the significant improvements in summarization techniques with the emergence of LLMs. It analyses the performance of various LLMs specifically MPT-7B-Instruct, Falcon-7B-Instruct, and OpenAI's text-davinci-003, in the context of text summarization. Results from experiments conducted on the CNN/Daily Mail and XSum datasets indicate that text-davinci-003 consistently outperforms the other models; its superior output quality, coherence, and instruction-following abilities makes it better owing to its large token capacity. MPT-7B-Instruct, fine-tuned on datasets like Dolly-15k and HH-RLHF, excels in short-form instruction tasks. Falcon-7B-Instruct, developed by TII, is an open-source model optimized for both chat and instruction-based tasks. This paper includes future works which includes testing larger models like MPT-30B-Instruct and Falcon-40B-Instruct, exploring larger and more diverse datasets and fine-tuning for domain-specific summarization tasks.

D. REVIEW ON METHODS USING LARGE LANGUAGE MODELS FOR CHESS COMMENTARY GENERATION

The work done by Jhamtani et al. [2] provides natural language commentary involving a large dataset of 298 K chess move-commentary pairs from 11K chess games, collected from online forums, which is significant in contrast to the traditional reliance on rule-based methods due to lack of large datasets. The paper deals with the challenges faced in chess commentary generation involving diverse commentary styles and dependence on pragmatic context. Their approach consists of two models: the end-to-end Neural Model, the Game-Aware Commentary (GAC) Model to generate meaningful commentaries. Chess game features (for example - threats, score, move type, etc.) are encoded using GAC-sparse and learned embeddings (GAC), a bidirectional LSTM (BiLSTM) encoder is used to model combinations of features dynamically, followed by the prioritization or selection of most relevant features which are then decoded by a recurrent LSTM decoder resulting in the generation of insightful commentaries in natural language. The research

done by Kim et al. [14] suggests that AI models, despite excelling in decision making, still lack interpretability for humans. LLMs (like GPT-4) are capable of generating fluent commentary but can be factually incorrect due to weak chess reasoning, commentary focuses on various aspects of a move and hence requires deep chess knowledge and linguistic skills. This paper, therefore, involves the usage of Concept-Guided Chess Commentary (CCC) as follows: (a) LLMs are guided via extracting significant chess concepts from expert models (like Stockfish, Leela Chess), (b) focus is on highly strategic and tactical ideas for better commentary, (c) few-shot learning and Chain-of-Thought prompting to enhance the model with minimal examples so that it adapts and learns to reason quickly even if there isn't enough data to fine tune a model. GPT-Based Chess Commentary Evaluation (GCC-Eval) uses traditional metrics like BLEU, ROUGE or METEOR which are not so effective because of their reliance on similarity to a reference text where they compare text based on word overlap. Commentary is subjective wherein different commentators carry different perspectives. Therefore, GCC leverages LLMs with expert model guidance to assess commentary on multiple dimensions. It integrates analysis from chess engines to check for strategically accurate comments instead of word overlapping and assesses the commentary for the reasoning behind a move. Moreover, according to experiments, GCC-Eval scores correlate strongly with human ratings of commentary quality.

The novel approach by Lee et al. [3] comes up with drawbacks of LLMs like GPT-4 which struggle with complex reasoning and real world grounding while still being able to generate fluent texts whereas the symbolic AI models (chess engines) excel in strategical, tactical and logical reasoning but human-like explanations serves as their downside. Each of the two models cover up for the other suggesting a combination of these two AI paradigms for an insightful chess commentary generation requiring rational, human-like explanations. Their approach is based on control codes (tags), which are extracted from the chess engines holding data such as move evaluation, tactical and positional concepts. The LLM is then trained based on these control tags rather than feeding them with raw output from the chess engines. This cuts down the burden on the LLMs as it is no longer required to understand chess deeply based upon its limited domain knowledge, instead it only needs to map control codes that align with expert-level understanding to natural language explanations. The key findings included better interpretability of moves, reduced reasoning errors and even the human judges preferred this model over previous baselines but unseen game positions and handling long-term strategies pose a problem, there's no complete eradication of reasoning errors as well. Recent approaches of retrieval-augmented commentary generation have also explored the use of historical game records and external databases to improve factual grounding and reduce hallucinations but it improves contextual consistency by leveraging previous gameplay patterns, the performance remains dependent on retrieval

quality and database coverage. Soliman et al. [15] proposed Caïssa AI, it is a neuro-symbolic chess agent which is designed for explainable move suggestion and grounded commentary generation. It is an integration of a chess-specific LLM with symbolic reasoning modules, Prolog-based tactical rule encoding, and a dynamically constructed knowledge graph representing the board state. The generated outputs are cross checked against symbolic logic through a verification module in order to reduce hallucinations and improve consistency in commentary. The study highlights the significance of integrating formal reasoning with language generation for trustworthy and interpretable chess narration. However, it primarily focuses on neuro-symbolic verification and rule-based grounding mechanisms whereas the proposed work follows reinforcement learning-guided contextual preprocessing and structured commentary annotations to support real time commentary generation. The study by Oshri et al. [16] takes up the neural chess engine enhanced approach to design automated generation model addressing five sub-tasks, namely, (a) Description (about the current move), (b) Quality (evaluation of move strength), (c) Contexts (analysis of the game situation for both sides), (d) Comparison (comparing different possible moves), (e) Planning (guessing the player's strategy). The model instead of relying on the human commentary dataset infuses a neural chess engine which can analyze the board, predict reasonable moves, evaluate different strategies through self-play. The Chess Engine is in turn pre-trained to not just memorise past games but have a deep understanding using: (a) Supervised learning from expert human games (FICS database) (b) Unsupervised self-play training (similar to AlphaZero). The system comprises an internal chess engine, move encoder (converts move to a form understandable by the language model), multi-choice encoder (permits multiple moves evaluation for comparative analysis). The qualitative aspect of the commentary is dependent on the playing strength of the neural chess engine. The approach by Toshniwal et al. [17] proposes a transformer model for chess move prediction; it can learn to track piece locations, predict legal chess moves, encode board state implicitly without extra probing mechanisms based on the training data. In place of traditional chess engines, transformer models can generate legal moves and keep track of board state from notations themselves, suggesting another potential approach of training language models on move sequences for chess commentary generation instead of relying on rule-based chess engines. Unlike LSTMs, it can handle additional input tokens with no loss in performance, transformer robustness to extra tokens help in suggesting new ways to inculcate extra knowledge without fine-tuning the entire model. Explicit board state tokens improves the accuracy of piece-tracking. Transformers can learn chess move legality, but performance drops significantly if history is truncated. The approach by Swingle et al. [18] also talks about Chessformer and how it leverages global self-attention for better long-range move

dependencies unlike AlphaZero's convolutional networks. ChePT-2 reframes chess as a language modeling problem permitting both move generation and natural language commentary. Yet again, there's a comparison between Chessformer and Alphazero with the former being more efficient requiring 8x less computation while matching grandmaster-level play with 30x less computation. ChePT-2 faces challenges in the late game due to limited dataset exposure but performs well in early and mid-game situations, its ability to generate reasonable and valid chess commentary improves upon the previously existing models concluding in increased relevance in commentary and reduced grammatical errors. Additionally, the model's chess strength slightly decreases when commentary is introduced, suggesting a tradeoff between gameplay strength and natural language generation capacity.

E. REVIEW ON REINFORCEMENT LEARNING FOR STRATEGIC COMMENTARY GENERATION

Reinforcement Learning (RL) has played a significant role over the time in enhancing strategic decision making systems in games and tasks related to language generation. AlphaZero combined with Monte Carlo Tree Search (MCTS) demonstrates how self play RL, without manual heuristics, can achieve superhuman chess performance. Likewise, MuZero extends this paradigm by focusing on internal representations of game dynamics directly from interactions. The Reinforcement Learning based optimization strategies motivate the proposed intermediary framework, where chess evaluations are converted into structured commentary tokens before final language synthesis. The proposed method, in contrast to purely retrieval based approaches, combines RL guided strategic abstraction with generation of natural language to enhance and improve both commentary coherence and contextual accuracy.

Schrittwieser et al. [19] proposed MuZero, a reinforcement learning framework that combines tree based search with learned environment modeling so as to achieve efficient and accurate performance across Atari games, chess, Go, and shogi without knowing the game dynamics explicitly. Unlike traditional planning systems, MuZero learns representations relevant for decision making, including estimation of policy, prediction of reward and evaluation of value directly from gameplay events. The study demonstrates effectiveness of learned planning mechanisms in complex sequential environments, reducing dependence on domain-specific engineering. In contrast, our approach leverages structured engine analysis and contextual annotations to convert strategic game insights into human-like commentary generation. Silver et al. [20] introduced AlphaZero, a generalized RL framework capable of learning chess, shogi, and go completely through self play without relying on domain specific evaluation functions or specific heuristics. The model integrates deep neural networks with Monte Carlo Tree Search (MCTS) in order to learn decision making directly from gameplay, interaction, user experience and achieving great performances

across multiple games. The work shows advantages and effectiveness of RL in long term planning and positional understanding using only game rules as prior knowledge. Li et al. [21] proposed RobustZero, it is a robust MuZero based RL framework which under adversarial and noisy state perturbations improves the policy consistency. This framework integrates adaptive policy adjustment mechanisms and self supervised representation learning to stabilize decision making in perturbed environments. Experimental evaluations were compared to existing baselines under both worst case and random perturbations, and it showcased improved robustness. However, the focus is on policy stability and robustness in RL tasks rather than contextual language generation. Existing RL frameworks primarily have its focus on gameplay optimization, policy robustness, and environment interaction. However, these approaches are not designed for explanatory narration or audience oriented interpretation of gameplay. The proposed framework extends reinforcement learning beyond optimization by using engine driven analysis with structured commentary annotations, enabling the transformation of complex positional evaluations into interpretable and coherent natural language commentary suitable for real-time spectator engagement. Overall, the existing approaches deal with gameplay optimization, symbolic reasoning, retrieval-based grounding or multimodal commentary generation independently but strategic accuracy, contextual coherence and real-time understanding simultaneously remains challenging. These limitations motivate the proposed framework integrating reinforcement learning guided preprocessing and structured human-centric chess commentary synthesis.

III. METHODOLOGY

This section provides an overview of the approach and subsequently explores each step in detail.

A. OVERVIEW

The approach first involves the pre-processing and structuring of the data input which is provided in the form of Portable Game Notation (PGN). PGN is a common representation of chess games and is how chess games are stored in most popular online chess game databases. These PGN files are obtained and then restructured for further processing. For the restructuring process, the PGN is converted into a game object stored as a JSON file. This game object is made of multiple nested move objects. This high-quality expansion of the original game data serves as the foundation for the further proposed preprocessing steps. It includes many important factors and fields such as the Forsyth–Edwards Notation(FEN) of the game state of the current move, the current move evaluation as generated by the Stockfish Chess Engine, the predicted best moves and the predicted best continuation as generated by the Stockfish Chess Engine, and other metadata for each move. This basic structure is passed into a special intermediate preprocessing model that is unique to the proposed approach. This model aims to add additional annotations and metadata, which can help

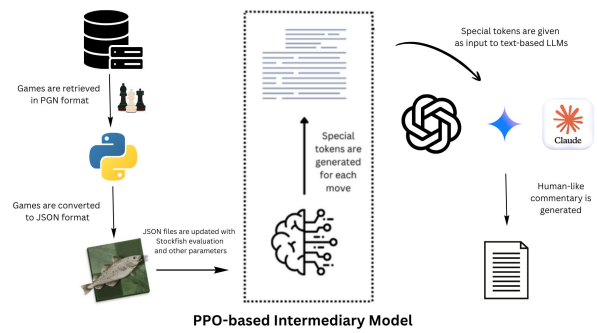


FIGURE 1. Diagrammatic visualisation of the proposed workflow.

enrich the context for the LLMs that will be generating the final move-by-move commentary in the further steps. The intermediate model interprets the move data from the previous step to add additional information such as the stage of the game (game in its opening stage, middle game stage, or endgame stage), the quality of a move (good or bad move), the impact of a move and other such information that can help the commentary model generate better commentary. We refer to these annotations as informational ‘special tokens’ that give the extra contextual information to the commentator model. The structured game and move objects along with the annotations/special tokens from the intermediate model are given as the input to the final LLM, to generate the move-by-move commentary while keeping in context the additional information provided by the special tokens. The core functionality of the proposed approach is visualised for simplicity in Fig. 1.

B. EXPERIMENTAL SETUP

This paper proposes a novel architecture for AI-based chess game commentary generation as discussed in the previous section. To test the performance of the proposed architecture, we retrieved numerous famous chess games of globally famous chess players to form the initial database of PGN games for processing and generating commentary. The chess module for Python was extensively used in the codebase for format conversion of the games (from PGN to JSON), to generate the FEN for each move of the game and to analyse the state of the game as part of the intermediary model. The entire preprocessing and restructuring was carried out in a local VSCode environment with Python 3.12.3. Stockfish Chess Engine was used to retrieve a move-by-move evaluation for the games. A publicly available API endpoint was used to generate the Stockfish predictions and continuations. For the commentary generation, the study explores two alternatives and compares their performance. We use OpenAI’s GPT-4o model and access it using the API provided by OpenAI. The prompting and output generation is done in the same VSCode environment as the previous step. We have also used the gemma4:e2b model which is part of the Gemma 4 family of models, locally. The commentary generation experiment with the fine-tuned Falcon model was

TABLE 1. Experimental setup for chess commentary generation.

Component	Details
Dataset Source	Famous chess games from www.chessgames.com and www.pgnmentor.com/files.html
Data Format	PGN converted to JSON and FEN using python-chess module
Preprocessing Environment	Local VSCode setup with Python 3.12.3
Game Evaluation	Stockfish Chess Engine via a public API for move-by-move evaluation
Commentary Models	GPT-4o (via OpenAI API), gemma4:e2b
Model Framework	GPT-4o accessed via API in VSCode; gemma4:e2b model used via Ollama on local machine

TABLE 2. PGN metadata interpretation and necessity.

PGN Heading	Interpretation	Necessity
Event	Tournament or event name	Helps provide context to the commentator
Site	Location	Can influence commentary
Date	Date of the game	May reflect player's form, preparation
EventDate	Start date of the event	Helps align the game chronologically
Round	Match round number	Situates the game in the event's structure
Result	Final result	Crucial for post-game analysis
White	Player with White pieces	Useful for analysis, player profiling
Black	Player with Black pieces	Useful for analysis, player profiling
ECO	Opening code (ECO)	Aids in technical commentary
WhiteElo	White's Elo rating	Gives insight into player strength
BlackElo	Black's Elo rating	Gives insight into player strength
PlyCount	Number of half-moves	Estimates game length

conducted on Google Colaboratory equipped with NVIDIA T4 GPUs. The experimental setup is elucidated in Table 1 as well.

C. GAME DATASETS

Generally, chess games are stored in the Portable Game Notation (PGN) format. This is a standard, concise, plain-text format of storing chess game information, which includes the Standard Algebraic Notation (SAN) to record moves and metadata like player names, event name, player rating, final result of the game, etc. Listing 1 shows an example game notation. 500+ chess games of famous players such as Vishwanathan Anand, Magnus Carlsen, Gukesh Dommaraju, Garry Kasparov etc. from www.chessgames.com and www.pgnmentor.com/files.html to form the initial raw, unprocessed database of PGN games.

Table 2 describes the information provided by the metadata and how it aids in generating more contextual commentary.

```
1 [Event "Spassky - Fischer World Championship
  Match"]
2 [Site "Reykjavik ISL"]
3 [Date "1972.08.10"]
4 [EventDate "?"]
5 [Round "13"]
6 [Result "0-1"]
7 [White "Boris Spassky"]
8 [Black "Robert James Fischer"]
9 [ECO "B04"]
10 [WhiteElo "?"]
11 [BlackElo "?"]
12 [PlyCount "148"]
13
14 1. e4 Nf6 2. e5 Nd5 3. d4 d6 4. Nf3 g6 5. Bc4 Nb6
   6. Bb3 Bg7 7. Nbd2 O-O 8. h3 a5 9. a4 dxe5
   10. dxe5 Na6 11. O-O Nc5 12. Qe2 Qe8 13. Ne4
   Nbxa4 14. Bxa4 Nxa4 15. Re1 Nb6 16. Bd2 a4
   17. Bg5 h6 18. Bh4 Bf5 19. g4 Be6 20. Nd4 Bc4
   21. Qd2 Qd7 22. Rad1 Rfe8 23. f4 Bd5 24. Nc5
   Qc8 25. Qc3 e6 26. Kh2 Nd7 27. Nd3 c5 28. Nb5
   Qc6 29. Nd6 Qxd6 30. exd6 Bxc3 31. bxc3 f6
   32. g5 hxg5 33. fxg5 f5 34. Bg3 Kf7 35. Ne5+
   Nxe5 36. Bxe5 b5 37. Rf1 Rh8 38. Bf6 a3 39.
   Rf4 a2 40. c4 Bxc4 41. d7 Bd5 42. Kg3 Ra3+
   43. c3 Rha8 44. Rh4 e5 45. Rh7+ Ke6 46. Re7+
   Kd6 47. Rxe5 Rxc3+ 48. Kf2 Rc2+ 49. Ke1 Kxd7
   50. Rxd5+ Kc6 51. Rd6+ Kb7 52. Rd7+ Ka6 53.
   R7d2 Rxd2 54. Kxd2 b4 55. h4 Kb5 56. h5 c4
   57. Ra1 gxh5 58. g6 h4 59. g7 h3 60. Be7 Rg8
   61. Bf8 h2 62. Kc2 Kc6 63. Rd1 b3+ 64. Kc3
   h1=Q 65. Rxh1 Kd5 66. Kb2 f4 67. Rd1+ Ke4 68.
   Rcl Kd3 69. Rd1+ Ke2 70. Rc1 f3 71. Bc5 Rxg7
   72. Rxc4 Rd7 73. Re4+ Kf1 74. Bd4 f2 0-1
```

LISTING 1. Spassky–Fischer, Game 13, 1972 World Championship.

D. GAME PRE-PROCESSING

While the PGN format given above is interpretable for an individual with basic chess notation familiarity and is a widely accepted method to store and analyze games, we opted to convert the format to a JSON game object to elaborate on the contents of every single move, which is essential for move-by-move commentary generation. For each move, the Forsyth–Edwards Notation (FEN) is generated and given to the Stockfish Chess Engine (accessed through the public Stockfish API), which returns the evaluation of the current move, the best move following this move, and the continuation for the next few moves. These additions to the existing game data form the base for seamless commentary. A segment of the JSON format representation of the game depicted below is shown in Listing 2. This segment is a single move object which is part of the game object. A game is made up of many such moves.

Table 3 denotes the various elements of the processed move and their relevance.

E. INTERMEDIARY MODEL

The introduction of an intermediary model is the key innovation of the proposed methodology. This model ensures that the generated commentary is not only technically accurate but also emotionally engaging. It does so by providing additional context to the LLM that will generate the final commentary by providing the LLM with special tokens that act as annotations

```
1 {
2   "Move Number": 1,
3   "Player": "White",
4   "Move": "c4",
5   "FEN": "rnbqkbnr/.... /RNBQKBNR
6       b KQkq - 0 1",
7   "Previous FEN": "rnbqkbnr/.... /RNBQKBNR w
8       KQkq - 0 1",
9   "Stockfish Analysis": {
10     "success": true,
11     "evaluation": -0.37,
12     "mate": null,
13     "bestmove": "e5",
14     "continuation": "e5 Nc3 ... O-O dxc4",
15     "ponder": "Nc3"
16   },
17   "eval_diff": -0.37
18 }
```

LISTING 2. Pre-Processed Move 1: Spassky - Fischer, Game 6, 1972 World Championship.

TABLE 3. Move-wise metadata and its significance in AI commentary.

Field	Value	Importance for AI Commentary
Move Number	1	Identifies the game phase (opening, mid- dle game, endgame)
Player	White	Indicates move own- ership for personal- ized narration
Move	c4	Core input for com- mentary generation
FEN	rnbqkbnr/.../RNBQKBNR b KQkq-0 1	Represents board state for engine- driven analysis
Previous FEN	rnbqkbnr/.../RNBQKBNR w KQkq-0 1	Tracks change from previous state, enabling transition commentary
Stockfish Suc- cess	true	Confirms validity of evaluation for this move
Evaluation Score	-0.37	Indicates positional advantage or disadvantage
Mate in N Moves	None	Identifies forced wins; adds dramatic effect
Best Move	e5	Compares human vs engine choice for cri- tique
Continuation	e5 Nc3 Nf6 e3 Bb4 Nge2 O-O a3 Be7 d4 d6 Ng3 exd4 exd4 d5 Be2 Nc6 O-O dxc4	Provides likely future trajectory for narra- tive extension
Ponder Move	Nc3	Used to anticipate op- ponent strategy
Evaluation Diff	-0.37	Measures accuracy of the move compared to optimal

and describe any special points of note to be considered during the commentary generation. The intermediary model implements the Proximal Policy Optimization algorithm, which generates special tokens based on the valuation given by the Stockfish Chess Engine. These special tokens dictate the emotion and the excitement quotient expected in the

TABLE 4. Move annotation tokens annotated by the intermediary model and their significance.

Token	Description
[MOVE]	Basic description of the move.
[BEST_MOVE]	Stockfish's recommended best move.
[CONTINUATION]	Suggested continuation line from engine.
[BEST]	Move is optimal per Stockfish.
[EXCELLENT]	Very strong move, maintains or improves position.
[GOOD]	Solid, stable move but not optimal.
[INACCURACY]	Slightly worsens position; not critical.
[MISTAKE]	Significantly worsens position, but not fatal.
[BLUNDER]	Leads to a major loss (material or mate).
[OPENING]	Move in first 10 turns.
[MIDGAME]	Move after 10th; midgame phase.
[ENDGAME]	Few pieces/pawns/majors; endgame criteria.
[CASTLING]	Castling detected via FEN change.
[EXCHANGE]	Capture identified by piece map diff.
[FORCING]	Move puts king in check.



FIGURE 2. Illustrative position to show pre-processing output.

generated commentary. When a game is given as input to the model, it analyses the evaluation and gives the special token for the current move. The special tokens of all the moves of a game are then compiled together into a JSON file, which is given as input to the fine-tuned model for commentary generation. Table 6 gives the list of special tokens annotated by the intermediary model.

To illustrate, we shall look at an example position shown in Fig. 2. The same is represented by the FEN r1b1k1nr/ppN1ppbp/2n3p1/1Bp5/4P3/5N2/PPP2PPP/R1B1K2R b KQkq-0 1.

In this position, the White player is in a significantly stronger position. In comparison to the Black player, the evaluation suggests an advantage of +3.96 (according to

Stockfish evaluation with depth=26) in favour for the White player. Listing 3 shows some representative annotative tokens that are generated by the intermediary model. Note that multiple intermediary tokens may be assigned to a single move. This is done to capture the complexity of the position using as many key indicators as possible. Each move will have the [MOVE], [BEST_MOVE] and [CONTINUATION] tokens ubiquitously. The [MOVE] special token denotes the move performed by the player. [BEST_MOVE] denotes the best move possible as suggested by Stockfish, while [CONTINUATION] represents the optimal trajectory, enhancing the context of narration generation. Based on the quality of the move, it may assign [BEST], [EXCELLENT], [GOOD], [INACCURACY], [MISTAKE] or [BLUNDER] as well. The quality of the move is decided by measuring the net change of the evaluation score (obtained from Stockfish engine) when a move is performed, which estimates the player that the current position is favoured towards. The intermediary model also assigns a token based on the stage of the game as well as for any special situations like exchanges, castling and forcing moves. In this specific example, there is no such special situation occurring.

```

1 [MOVE]
2 White plays Nc7+ from the position:
3 r1b1k1nr/ppN2ppp/2n5/1Bp5/
4 4P3/5N2/PPP2PPP/R1BQK2R
5 b KQkq - 0 1.
6 [/MOVE]
7
8 [BEST_MOVE]
9 Stockfish recommends: Nc7+.
10 [CONTINUATION] Stockfish suggests:
11 Nc7+ Kf8 Nxa8.
12
13 [EXCELLENT]
14 This is an excellent tactical move,
15 forking the king and rook.

```

LISTING 3. Final annotated move generated by the Intermediary Model.

F. PREPARING THE RL MODEL

The proposed commentary generation model implements a Reinforcement Learning (RL) paradigm, based on the Proximal Policy Optimization (PPO) algorithm. This algorithm dynamically annotates chess moves with semantically meaningful special tokens. PPO is an on-policy algorithm, which means that it learns from data that it collected using the current policy. Its main innovation is the introduction of a clipped objective function, which prevents the RL agent from making huge, destabilising updates. The implemented architecture employs an actor-critic structure in which the actor network learns a stochastic policy to predict tokens, while the critic network estimates the expected state value (value function) to stabilise the overarching policy updates. The PPO optimisation objective follows the clipped loss formulation, which constrains excessively large policy updates and stabilises training. The probability ratio between

new and old policies is computed as follows:

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \quad (1)$$

where π_θ is the new policy and $\pi_{\theta_{\text{old}}}$ is the original policy with which the model starts out with.

- If $r_t(\theta) > 1$, the action is more likely to be taken under the new policy compared to the old policy.
- If $r_t(\theta) = 1$, the new policy assigns the exactly same probability to the action as that of the original policy.
- If $0 < r_t(\theta) < 1$, the action is less likely to be taken under the new policy.

The clipped PPO objective is then defined as:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)] \quad (2)$$

where A_t denotes the advantage estimate, and ϵ is the clipping coefficient controlling the numerical magnitude of the update. In the implemented system, the clipping parameter was set to 0.2 to ensure stable policy improvement. The final optimisation loss combines policy loss, critic value loss, and entropy regularisation:

$$L = L_{\text{policy}} + 0.5L_{\text{value}} - \beta H(\pi_\theta) \quad (3)$$

where $H(\pi_\theta)$ represents policy entropy and β controls exploration strength. It dictates how much the agent explores its environment versus exploiting what it has already.

A key benefit of the proposed PPO method is its domain-aware reward strategy. The algorithm defines dense, semantic-based rewards derived from the chess engine evaluations and positional heuristics rather than depending totally on surface-level win/loss signals. The reward function evaluates whether the predicted special tokens correctly reflect the underlying chess position. Several reward components are incorporated:

- **Phase Consistency Reward:** Positive rewards are assigned when the predicted game phase aligns with deterministic phase detection heuristics.
- **Move Quality Reward:** Rewards are computed using changes in Stockfish evaluation scores between consecutive moves.
- **Best-Move Alignment Reward:** Additional rewards are granted when annotations such as [BEST] correspond to engine-recommended moves.
- **Tactical Context Reward:** Tactical annotations such as [FORCING] are rewarded when the resulting board state places the opponent in check.
- **Penalty Mechanisms:** Strong penalties are imposed for semantically impossible predictions, such as labelling early opening positions as endgames.

Following policy inference, the predicted tokens are transformed into natural-language commentary templates. Structural commentary tokens such as [MOVE], [BEST_MOVE], and [CONTINUATION] are always generated, while

PPO-predicted annotations augment the explanation with qualitative insights regarding move strength and positional context. This pipeline enables the generation of coherent, context-aware, and strategically meaningful chess commentary. Overall, the proposed PPO-based framework demonstrates how reinforcement learning can be effectively adapted for semantic annotation tasks in chess analysis. By combining structured board representations, domain-aware reward shaping, stable PPO optimisation, and deterministic chess heuristics, the system achieves a robust balance between strategic accuracy and linguistic coherence in automated chess commentary generation.

The board state $s \in \mathbb{R}^{73}$ at each ply is constructed via a deterministic encoding function $\phi(\cdot)$ applied to the Forsyth–Edwards Notation (FEN) of the position. The actor network π_θ employs a shared two-layer Multi-Layer Perceptron (MLP) backbone ($73 \rightarrow 256 \rightarrow 128$, ReLU activations). The critic network V_ϕ shares the same backbone architecture ($73 \rightarrow 256 \rightarrow 128 \rightarrow 1$, linear output) and estimates the state-value $V(s)$ for advantage computation.

Further details of the PPO algorithm implementation can be found in our GitHub repository 1

IV. GENERATION OF OUTPUT COMMENTARY

A. EVALUATION METRICS

We developed a Multi-Dimensional Evaluation Framework with 7 complementary metrics to quantify the quality of machine-generated chess commentary. The framework consists of a Tier 1 and a Tier 2. Tier 1 contains 4 *primary* metrics that are closely aligned with the linguistic and semantic quality of the text generated and Tier 2 contains 3 *auxiliary* metrics that measure the structural integrity, tactical fidelity, and textual diversity of the text generated. Each of the 7 metrics is normalized to the range [0, 100] to allow for direct comparison and aggregation across metrics.

The following formula presents the primary composite score associated with move m :

$$S_{\text{primary}}(m) = \frac{1}{4} \sum_{k \in \mathcal{P}} s_k(m) \quad (4)$$

where $\mathcal{P} = \{\text{Fluency, Correctness, Informativeness, Insightfulness}\}$ and $s_k(m) \in [0, 100]$ shows the normalized score of metric k at move m .

All seven metrics along with their formal definition, computation methods and the means of interpreting boundary values can be found in the Table 5.

1) METRIC VISUALIZATION AND INTERPRETATION

In order to conduct longitudinal analysis of the quality of commentary, we will plot the primary composite score $S_{\text{primary}}(m)$ as a function of move number m for each evaluated model to see how they progress throughout the course of a game over time; game phases (opening, middlegame, endgame) are represented by shaded areas on the timeline so that model performance can be viewed by phase. Instead of displaying multiple timelines (one for each of the per-metric

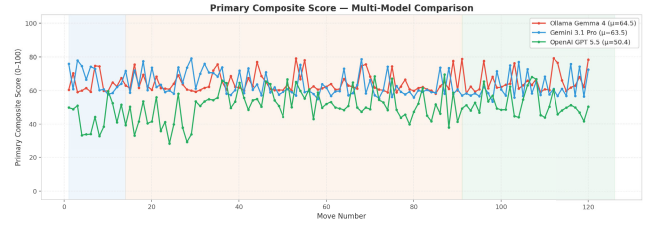


FIGURE 3. Primary composite score trajectory across move numbers for the three evaluated Large Language Models (LLMs). Dashed horizontal lines indicate per-model means. Shaded regions denote game phases: Opening (blue), Middlegame (orange), and Endgame (green).

timeline) due to limited website space, we chose to create a unified set of graphs with a composite trajectory together with mean reference lines for each model (dashed lines). This allows easy comparison between the temporal behaviors of each of the models visually.

Figure 3 shows that the composite trajectory also reveals model-specific patterns of behaviour: High-quality, sustained commentary manifests as stable, high level traces; on the other hand, trace patterns that are unstable or decreasing will demonstrate weaknesses that can be attributed to phase dependent characteristics. Frequently, sudden drops in score for specific moves coincide with complex tactical situations where the Language Model(LM) did not provide commentary that is fluent or semantically consistent with the rating of the annotated move.

B. COMMENTARY SOURCES AND SYSTEM ARCHITECTURE

A two-part pipeline for generating chess commentary exists which outputs both token-level move articles from unstructured chess engine data using a reinforcement learning (RL) intermediary system. This output is then inputted into three independent lower dimensional language models (LLM) that convert these token-level moves into fluent English commentaries. This decoupled approach provides the ability to control for the effect of model on commentary quality by providing the same RL outputs as input for all three language model systems.

1) STAGE 1: REINFORCEMENT LEARNING INTERMEDIARY MODEL

The intermediate model is an agent utilizing Proximal Policy Optimisation (PPO) [22] to annotate moves within a chess game with the specified number of intermediate tokens, producing a 73-dimensional state representation $s \in \mathbb{R}^{73}$ at each ply from the current board position:

- 64 dimensions encoding the piece map (piece type $\times$ colour per square),
- 1 normalised move number (ply/100),
- 4 binary castling rights (White kingside/queenside, Black kingside/queenside),
- 1 binary in-check flag,
- 1 normalised material balance ((White – Black)/39),
- 1 normalised non-king piece count (piece count/30),

TABLE 5. Evaluation metrics: Definitions, computational methods, and score interpretation.

Metric	Tier	Computational Method	High Score (≥ 75)	Low Score (≤ 30)
Fluency	Primary	GPT-2 perplexity (PPL) mapped via logistic function, combined with Flesch-Kincaid (FK) readability grade. $0.65 \times \text{PPL}_{\text{score}} + 0.35 \times \text{FK}_{\text{score}}$.	Grammatically correct, natural prose at an appropriate reading level for chess commentary.	Incoherent, fragmented, or syntactically malformed text with very high perplexity.
Correctness	Primary	DeBERTa-v3 Natural Language Inference (NLI) entailment probability against quality-token hypothesis, combined with token confidence. $0.70 \times \text{NLI}_{\text{entail}} + 0.30 \times \text{Conf.}$	Commentary accurately reflects the quality assessment (e.g., labels a blunder as a blunder).	Commentary contradicts the annotated move quality or is semantically inconsistent with the token.
Informativeness	Primary	Measure of Textual Lexical Diversity (MTLD), unique n -gram ratio (bigram and trigram), and word count. $0.35 \times \text{MTLD} + 0.35 \times \text{Ngram} + 0.30 \times \text{Length}$.	Rich, diverse vocabulary with substantial explanatory content and adequate length.	Repetitive word choices, minimal lexical variation, and insufficient explanatory depth.
Insightfulness	Primary	Sentence-BERT (S-BERT) cosine similarity to a curated corpus of 20 expert reference commentaries, combined with strategic concept density. $0.60 \times \text{Sim}_{\text{top3}} + 0.40 \times \text{Concept}$.	Commentary exhibits expert-level strategic reasoning and uses domain-specific terminology.	Generic or superficial observations with no strategic depth or domain vocabulary.
Factual Consistency	Auxiliary	Intra-commentary DeBERTa-v3 NLI contradiction detection across sentence pairs. Score = $(1 - \max(\text{contradiction})) \times 100$.	Internally consistent commentary with no self-contradictory statements.	Commentary contains logically contradictory claims within the same move annotation.
Tactical Accuracy	Auxiliary	DeBERTa-v3 NLI entailment against tactical event hypotheses (e.g., exchange, sacrifice, castling). Average entailment across detected tactical tokens.	Commentary accurately describes the tactical events that occurred in the position.	Commentary fails to describe or mischaracterises the tactical events present.
Redundancy	Auxiliary	Self-BLEU (inter-move similarity) and zlib compression ratio. $0.50 \times (1 - \text{SelfBLEU}) + 0.50 \times \text{CompressRatio}$. Higher score indicates lower redundancy.	Unique, non-repetitive commentary with high information density across moves.	Highly templated or copy-paste commentary with minimal variation between moves.

- 1 normalised queen count (queen count/2).

For the actor network, all three independent prediction heads employ a single-shared two-layer multi-layer perceptron (MLP) backbone ($73 \rightarrow 256 \rightarrow 128$, ReLU activations):

- **Quality head** ($128 \rightarrow 6$, softmax): Produces a categorical distribution over {[BEST], [EXCELLENT], [GOOD], [INACCURACY], [MISTAKE], [BLUNDER]}.
- **Phase head** ($128 \rightarrow 3$, softmax): Produces a categorical distribution over {[OPENING], [MIDGAME], [ENDGAME]}.
- **Context head** ($128 \rightarrow 1$, sigmoid): A Bernoulli gate for the [FORCING] token.

For the critic network, the critic estimates the state-value function $V(s)$, using an identical backbone but producing a single linear output. Training occurs for 5,000 epochs per game utilizing the clipped PPO objective having $\gamma = 0.99$, $\epsilon_{\text{clip}} = 0.2$, learning rate of 3×10^{-4} (Adam optimizer), entropy coefficient 0.02, and gradient clipping at $\|\nabla\| = 0.5$.

The reward function has four parts: (i) phase accuracy (+2.0 for correct phases with penalties for impossible predictions), (ii) quality accuracy (+3.0 if it exactly matches

the Stockfish oracle with graded penalties proportional to the ordinal distance), (iii) best-move alignment (+1.5 if the move played matches the recommendation from the engine and the quality token reflects this) and (iv) forcing-move gating (+1.0 if [FORCING] is correctly emitted in check positions and -1.0 for false positives). Importantly, during inference, a deterministic oracle uses Stockfish evaluation differences and board-state heuristics to overwrite the PPO actor's quality and phase predictions, using PPO's confidence scores as a secondary signal. This hybrid method ensures that the annotations are correct while also maintaining the distribution of large confidence that has been learned. Table 6 shows all tokens generated by the intermediate model.

2) STAGE 2: DOWNSTREAM LARGE LANGUAGE MODELS

The structured token-annotated output from the RL intermediary model is provided as contextual input for three different LLMs all will generate fluent, natural language chess commentary on an annotated chess game that is same for all of them. All LLMs share a common annotated game file to ensure a controlled experimental layout.

- 1) **OpenAI GPT-4o / ChatGPT [23]**: Accessed via OpenAI API and interfaces with the entire annotated move sequence to produce detailed multi-sentence

TABLE 6. Move annotation tokens produced by the intermediary RL model and their significance.

Token	Description
<i>Structural Tokens (Deterministic)</i>	
[MOVE]	Basic description of the played move (player, UCI notation).
[BEST_MOVE]	Stockfish's recommended best move for the position.
[CONTINUATION]	Engine principal variation (continuation line).
<i>Quality Tokens (PPO + Oracle)</i>	
[BEST]	Objectively the best move; matches engine top choice.
[EXCELLENT]	Very strong move, close to engine recommendation ($\Delta \geq 0.3$).
[GOOD]	Solid move maintaining positional balance ($\Delta \geq -0.2$).
[INACCURACY]	Slight inaccuracy; better continuation available ($\Delta \geq -0.5$).
[MISTAKE]	Significant error worsening the position ($\Delta \geq -1.5$).
[BLUNDER]	Critical blunder; evaluation swings sharply ($\Delta < -1.5$).
<i>Phase Tokens (PPO + Heuristic Override)</i>	
[OPENING]	Development and centre control phase (≤ 3 undeveloped minor pieces, ply ≤ 30).
[MIDGAME]	Complex middlegame with full piece activity.
[ENDGAME]	Simplified position (≤ 7 pieces per side, or ≤ 1 major piece per side).
<i>Event Tokens (Deterministic Detection)</i>	
[CASTLING]	King castles (detected via UCI pattern: e1g1, e1c1, e8g8, e8c8).
[EXCHANGE]	A piece capture occurs (detected via board replay).
[FORCING]	Move delivers check (gated: requires PPO agreement > 0.3).

commentary for each move. Produces well-organized, verbose, strong following of instructions and has a very high-quality output.

- 2) **Google Gemini 2.5 Pro [24]**: Accessed via Google AI studio API, uses Gemini to process the annotated input and generate a balanced level of technical detail and contextually relevant commentary about the events happening within the game as it is being played (as represented in their respective phases).
- 3) **Ollama Gemma 4**: Deployed locally via the Ollama inference framework and will function in completely offline mode. Will produce longest length of commentary output on average, with the greatest amount of lexical diversity, will include a high-level of positional analysis; however, there may be more verbose than ChatGPT or Gemini.

Each LLM will produce a JSON array output in the following schema will be disambiguated as: Move Number, Intermediate Tokens (preserved from the RL model), Confidence (preserved), and Generated Commentary (the LLM-produced natural-language text). All outputs adhere to a standardised format, which will allow

TABLE 7. System specifications for model training and evaluation.

Component	Specification
Operating System	Windows 11
Runtime	Python 3.11, PyTorch 2.x (CPU)
PPO Training	5,000 epochs, Adam ($\text{lr} = 3 \times 10^{-4}$)
Chess Engine	Stockfish (depth 20, multi-PV)
Fluency Model	GPT-2 (124M parameters)
NLI Model	DeBERTa-v3-small (cross-encoder, NLI)
Embedding Model	all-MiniLM-L6-v2 (S-BERT, 384-dim)
LLM (Cloud)	OpenAI GPT-4o, Google Gemini 2.5 Pro
LLM (Local)	Ollama Gemma 4
Dataset	75 games (Anand, Gukesh, Fischer, Carlsen)
Statistical Tests	Wilcoxon signed-rank, Cohen's d (SciPy)

direct consumption by the evaluation pipeline outlined in Section IV-C.

3) SYSTEM SPECIFICATIONS

Table 7 shows in detail the hardware and software configurations used for all experiments.

C. EVALUATION METHODOLOGY AND PIPELINE

The evaluation framework is completely automated, with inputs coming from structured commentary in JavaScript Object Notation (JSON) format; then this information is processed using various pre-trained transformer models and established Natural Language Processing (NLP) metrics to yield per-move scores, game summaries, and visualization output for publications. The pipeline is agnostic to which source of commentary it works with; the only requirement for any given entry into the pipeline is that each must conform to a standard schema containing four fields: Move Number, Intermediate Tokens, Confidence, and Generated Commentary.

1) DATA INGESTION AND PREPROCESSING

Each input JSON contains all moves of one game in an ordered array made up of move-level entries. Once the pipeline has received the input JSON, it will check for schema conformity and then extract three sets of intermediate tokens as produced by the upstream Reinforcement Learning (RL) model:

- **Quality tokens**: A mutually exclusive categorical label from the set {[EXCELLENT], [GOOD], [INACCURACY], [MISTAKE], [BLUNDER]}, reflecting the move's engine-assessed quality.
- **Phase tokens**: A single game-phase indicator from {[OPENING], [MIDGAME], [ENDGAME]}, defined by board-state heuristics.
- **Tactical tokens**: Zero or more event descriptors from the set {[EXCHANGE], [SACRIFICE], [CASTLING], [CHECK], [FORK], [PIN], ...}, detected deterministically from the position.

2) FEATURE EXTRACTION AND MODEL INFERENCE

Three pretrained neural network models are used to implement the pipeline, with a lazy loading strategy to keep memory requirements to a minimum.

- 1) **GPT-2 [25]**: The generator's output token-level perplexity score is determined, and then the raw perplexity value PPL is converted into a normalised score through the use of a logistic function. This function is anchored at $\text{PPL} = 150 \rightarrow 50$:

$$s_{\text{PPL}} = \frac{100}{1 + e^{0.02 \cdot (\text{PPL} - 150)}} \quad (5)$$

- 2) **DeBERTa-v3-small (NLI) [26]**: A natural language inference (NLI) cross-encoder that has been fine-tuned on two corpora (Multi-Genre NLI / MultiNLI and Stanford NLI / SNLI) provides a probability distribution (over {entailment, neutral, contradiction}) for premise-hypothesis pairs. This model is used for the following three distinct scoring methods:

- *Correctness*: The commentary (premise) will be evaluated against a quality-token-specific hypothesis (e.g., "This was an excellent and brilliant chess move" for [EXCELLENT]).
- *Factual consistency*: All of the intra-commentary sentence pairs (s_i, s_j) will be cross-evaluated, and the maximum contradiction score will determine the consistency score.
- *Tactical accuracy*: The commentary will be evaluated against "tactical-event" hypotheses (e.g., "A material sacrifice was made to gain positional or tactical advantage" for [SACRIFICE]).

- 3) **Sentence-BERT (S-BERT) [27]**: Using the all-MiniLM-L6-v2 architecture 384-dimensional semantic space shared by all commentaries in the game and batch-embedded, the commentaries' embeddings serve two purposes:

- *Insightfulness*: Cosine similarity is calculated using a curated corpus of 20 expert chess commentary references, covering structural, tactical, and endgame themes. The expert alignment score is obtained by taking the mean of the top three similarities.
- *Informativeness*: The embeddings give context for the lexical diversity computation by confirming that high Measure of Textual Lexical Diversity (MTLD) values correspond to genuinely diverse semantic content.

Additionally, Three other features are computed in addition to that:

- **Flesch-Kincaid grade [28]**: Readability oriented toward an optimal score of 8 through 14 for chess commentary.
- **MTLD [29]**: An index of lexical diversity, robust and independent of length, computed bidirectionally with a type-token ratio (TTR) threshold of 0.72.
- **Self-BLEU [30]**: Inter-move redundancy quantified via the BLEU-4 score of each commentary against all other

TABLE 8. Comprehensive comparative performance matrix across LLMs (Scores: 0–100).

Metric	Gemma 4	Gemini Pro	GPT-4o
Fluency	87.7	85.0	59.6
Correctness	45.1	52.1	50.5
Informativeness	87.1	78.8	54.2
Insightfulness	38.0	38.0	37.3
Factual Consistency	32.8	28.5	72.6
Tactical Accuracy	73.0	73.3	78.3
Redundancy	71.6	74.4	90.6
Primary Avg.	64.5	63.5	50.4

commentaries in the same game combined with the zlib compression ratio.

3) SCORE AGGREGATION AND REPORTING

For each move m , seven metric scores $s_k(m) \in [0, 100]$ are computed and then aggregated into:

- **Move-level**: A primary composite $S_{\text{primary}}(m)$ (Eq. 4) and all seven individual scores, exported as Comma-Separated Values (CSV) and JSON.
- **Game-level**: Phase-stratified averages (Opening, Middlegame, Endgame), best/worst move identification per metric, quality-token distribution, and diagnostic flags (e.g.: "fluent but factually wrong": fluency ≥ 70 and correctness ≤ 30).
- **Comparative-level**: Statistical significance is determined using the pairwise Wilcoxon signed-rank tests [31] and the Cohen's d effect sizes [32] for each metric in considering several models on the same game.

D. COMPARATIVE ANALYSIS AND EXPERIMENTAL RESULTS

To assess the effectiveness of all 3 downstream LLMs transforming RL-annotated chess data into quality natural language commentary, a controlled comparative experiment was performed. All 3 models, OpenAI GPT-4o, Google Gemini 2.5 Pro, and Ollama Gemma 4, were provided with the same PPO-annotated game, Gukesh Game (IIFL Wealth Mumbai Open), containing 120 half-moves; therefore, any performance differences in all cases can be attributed solely to each LLM's language generation capabilities since the upstream RL signal was made constant.

1) OVERALL PERFORMANCE SUMMARY

Table 8 provides the means through all 7 evaluation metrics for each LLM. Ollama Gemma 4 received the highest primary composite score ($\bar{S}_{\text{primary}} = 64.5$), followed by Gemini 2.5 Pro (63.5) and GPT-4o (50.4).

2) FLUENCY AND LINGUISTIC QUALITY

Compared to Gemini 2.5 Pro, Ollama Gemma 4 maintains significantly lower GPT-2 perplexity scores ($\text{PPL} < 100$) and comparable Flesch-Kincaid reading level scores in the optimal range of 8 – 14, achieving fluency ratings

of 87.7 and 85.0, respectively. A Wilcoxon signed-rank test confirmed statistically significant differences between the two (Wilcoxon $p < 0.001$, Cohen's $d = 0.635$, medium effect). In contrast, GPT-4o's fluency score (59.6) was affected by shorter commentary lengths leading to higher perplexity estimates and placement outside of the desired readability range. A large effect size ($d = 2.569$, $p < 0.001$) existed between the fluency scores of Gemma and GPT-4o.

3) CORRECTNESS AND SEMANTIC ALIGNMENT

In terms of the accuracy of responses generated via DeBERTa-v3 Natural Language Inference (NLI) entailment, using a criterion of quality Token Hypotheses, the landscape becomes increasingly nuanced revealed by Correctness. Gemini 2.5 Pro receives the highest rating (52.1), followed closely behind by GPT-4o (50.5) and Gemma (45.1), which represents statistically significant differences (Gemma-Gemini; $p = 0.016$, $d = -0.288$) and no statistically significant differences between the first two models (Gemini - GPT-4o; $p = 0.444$, negligible effect). This data suggests that lengthier more elaborate commentary provided by Gemma does not provide a semantic alignment with annotated Move Quality at a greater rate than models producing shorter more concise comments.

4) INFORMATIVENESS AND LEXICAL DIVERSITY

Gemma 4 achieved the highest rate of informativeness (87.1) by a considerable margin, attributed to consistently high MTLTD values (>100), high distributions of unique n-grams ($>99\%$) and average word usage per move ranging from 75 – 85 words. The effect of Gemma on its relative performance when compared to GPT-4o is the largest effect observed in the study ($d = 4.210$, $p < 0.001$). Conversely, the substantially lower level of lexical diversity and length associated with GPT-4o (20 - 40 words per move) ultimately resulted in this model receiving considerably lower scores than Gemma (54.2).

5) INSIGHTFULNESS AND EXPERT ALIGNMENT

Overall, all three models produced comparable levels of Insightfulness (score range: 37.3 to 38.0) resulting in no statistically significant differences in pairwise comparisons between models ($p > 0.33$ for all pairs, $|d| < 0.13$). This ceiling effect suggests that none of the evaluated Large Language Models (LLMs) produced commentary with a semantic alignment to a level consistent with Expert Strategic Reasoning (as determined via Sentence-BERT similarity to the curated reference corpus) thus representing a considerable area of study for further research.

6) AUXILIARY METRICS: FACTUAL CONSISTENCY, TACTICAL ACCURACY, AND REDUNDANCY

With regard to auxiliary metrics, all three auxiliary metrics indicate GPT-4o substantially outperformed its two competitors. For example, GPT-4o's factual consistency score of

72.6 significantly exceeds both Gemma's (32.8) and Gemini's (28.5), with large effect sizes ($d = -0.986$ and $d = -1.122$ respectively, both $p < 0.001$). An examination of how the resulting commentary for each of the three models compared with the original text provides an explanation of the higher factual consistency level for GPT-4o; shorter, more conservative commentary offers fewer opportunities for contradictory statements to occur within a single piece of commentary. GPT-4o's redundancy score of 90.6 likewise illustrates the uniqueness and non-templated nature of the commentary provided throughout all of its moves when compared with the similar sentence pattern and templated structure employed by Gemma, which received a score of 71.6 on this same measure.

7) PHASE-SPECIFIC ANALYSIS

Variation in performance by phase of play illustrates model-specific advantages:

- **Opening:** Gemini 2.5 Pro had the highest score of correctness in the Opening (62.6), substantially outperforming both Gemma (48.0) and GPT-4o (50.3), which demonstrates that the model is optimally configured and produce comments to correspond to well-established opening theory.
- **Middlegame:** Gemma produced the most fluent (87.8) and informative commentary (87.8) in the Middlegame where intricate tactical positions require detailed explanation.
- **Endgame:** All models produced similar quality results since there was no statistically significant difference in their primary scoring metrics; since there is less complexity associated with the simplified tactical position, this result is expected.

The end-to-end time complexity of the proposed pipeline is

$$\mathcal{O}\left(G\left[N\left(L^2 + (K^2 + T)S^2\right) + N^2W\right]\right),$$

where G denotes the number of games, N the number of moves per game, L the commentary token count, S the sentence-pair token count, K the number of sentences, T the number of tactical tokens, and W the commentary word count.

8) VISUAL SUMMARY

Fig. 4 shows the relative strengths and weaknesses of the three LLMs compared across the seven evaluation metrics. Fig. 5 also presents a bar chart style view of the same data for accurate numerical comparisons. Fig. 6 shows how per-metric scores are distributed across the three LLMs and each model's respective distributions for how they vary from other models' performances across each of the seven evaluation metrics.

Table 9 lists the results of the pairwise statistical significance tests across all the primary metrics.

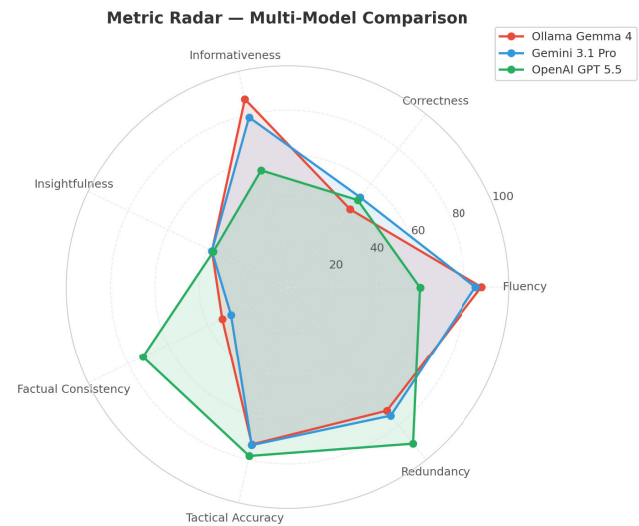


FIGURE 4. The radar chart shows the seven evaluation metrics as measured on each of the three LLMs: Gemma 4 from Ollama, Google Gemini 2.5 Pro, and OpenAI’s GPT-4o. Each axis represents a metric normalised to [0, 100].

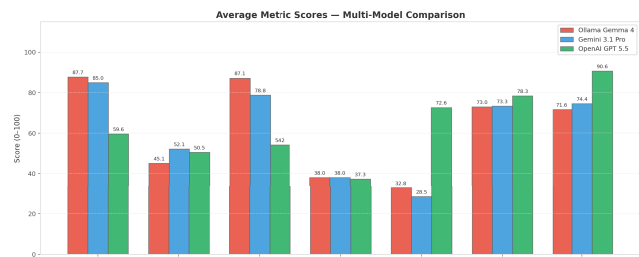


FIGURE 5. The mean evaluation scores across each of the three LLMs are shown in a grouped bar chart. Each bar consists of the mean performance value per model-metric pair for the three LLMs and the error bars provide the actual mean scores for performance for each model-metric pair.

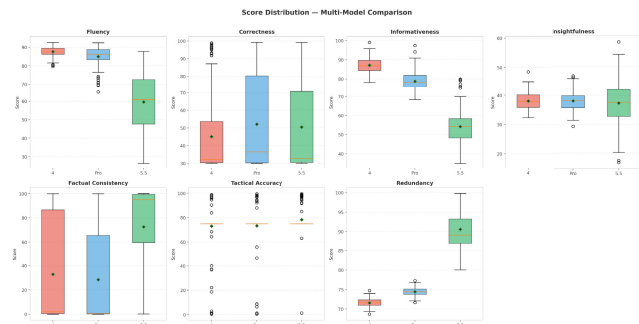


FIGURE 6. The box plot for each evaluation metric shows how scores per move were distributed for the three LLMs. The diamonds indicate the mean score for each LLM and the ends of the box show the range of the scores relative to the mean score (i.e., the range of the interquartile range (IQR)).

V. DISCUSSION

A. LIMITATIONS

One of the key limitations of the proposed approach is its reliance on a structured and annotated dataset. While this enables precise supervision, it may hinder the model’s ability

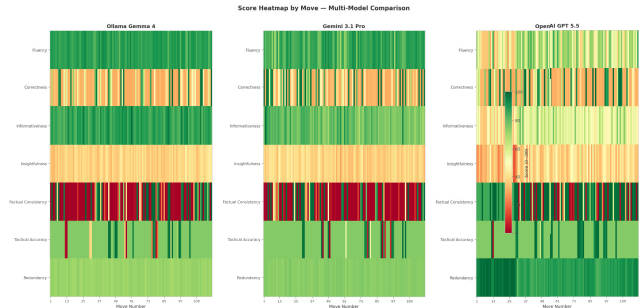


FIGURE 7. Score heatmap shows the intensities (metric scores) for every move of each language model. The y-axis displays the seven different metrics; the x-axis contains the move number, and the colour intensity will be from red through yellow and green to represent the corresponding score of that metric on a range [0, 100].

TABLE 9. Wilcoxon signed-rank test results and effect sizes (Cohen’s *d*) for the primary performance measures.

Metric	Pair	<i>p</i>	Sig.	<i>d</i>
Fluency	Gemma–Gemini	<0.001	***	0.64
Fluency	Gemma–GPT	<0.001	***	2.57
Fluency	Gemini–GPT	<0.001	***	2.22
Correctness	Gemma–Gemini	0.016	*	−0.29
Correctness	Gemma–GPT	0.094	ns	−0.22
Correctness	Gemini–GPT	0.444	ns	0.06
Informativeness	Gemma–Gemini	<0.001	***	1.88
Informativeness	Gemma–GPT	<0.001	***	4.21
Informativeness	Gemini–GPT	<0.001	***	3.00
Insightfulness	Gemma–Gemini	0.652	ns	−0.02
Insightfulness	Gemma–GPT	0.373	ns	0.12
Insightfulness	Gemini–GPT	0.338	ns	0.12

to generalize to commentary styles or games that fall outside the scope of the training data. The inclusion of a task-specific intermediary model to generate special tokens, although beneficial in enriching the training signal, introduces an additional layer of complexity. Errors in token generation or biases in the intermediary model could propagate to the final commentary output. The comparative evaluation with leading language models, although quite informative and insightful, is constrained by differences in model architecture, training scale, and accessibility, limiting a fully fair benchmark.

B. FUTURE WORK

Future extensions of this study involve broadening the data spectrum by incorporating unsupervised game commentary datasets. This will allow the model to develop a more nuanced understanding of diverse playstyles and commentary forms, enhancing both linguistic and contextual versatility. Another promising direction is the integration of a Retrieval-Augmented Generation (RAG) pipeline, which would enable the model to access and incorporate recent game data or updated opening theory in real-time. This could significantly improve the relevance and timeliness of generated commentary. Additionally, expanding the system to support multilingual commentary and potentially integrating speech synthesis could make the framework more accessible and engaging for global audiences. Optimizing

the intermediary token generation model for higher accuracy and reduced noise remains a key technical goal for future iterations.

VI. CONCLUSION

This work presents a reinforcement learning-guided framework for generating human-like chess commentary using structured chess engine analysis and contextual language generation. The proposed methodology proposes a PPO-based intermediary model that annotates the chess moves with strategically meaningful tokens, enabling language models to generate accurate commentary which is context-aware and coherent. By combining Stockfish evaluations, structured game representations, and task-specific annotations, the framework improves upon strategic interpretability and linguistic quality. Comparative analysis with GPT-4o, Gemini 2.5 Pro, and Gemma 4 demonstrates the effectiveness and accuracy of the proposed architecture in generating engaging, human-like, strategically grounded chess commentary.

VII. SOURCE CODE AND SCRIPTS

The source code and detailed instructions required to reproduce the experiments presented in this paper are publicly available.¹

REFERENCES

- [1] H. Kameko, S. Mori, and Y. Tsuruoka, "Learning a game commentary generator with grounded move expressions," in *Proc. IEEE Conf. Comput. Intell. Games (CIG)*, Aug. 2015, pp. 177–184.
- [2] H. Jhamtani, V. Gangal, E. Hovy, G. Neubig, and T. Berg-Kirkpatrick, "Learning to generate move-by-move commentary for chess games from large-scale social forum data," in *Proc. 56th Annu. Meeting Assoc. Comput. Linguistics*, 2018, pp. 1661–1671.
- [3] A. Lee, D. Wu, E. Dinan, and M. Lewis, "Improving chess commentaries by combining language models with symbolic reasoning engines," 2022, *arXiv:2212.08195*.
- [4] S. Maharaj, N. Polson, and A. Turk, "Chess AI: Competing paradigms for machine intelligence," *Entropy*, vol. 24, no. 4, p. 550, Apr. 2022.
- [5] J. Baxter, A. Tridgell, and L. Weaver, "Learning to play chess using temporal differences," *Mach. Learn.*, vol. 40, pp. 243–263, May 2000.
- [6] D. Noever, M. Ciolino, and J. Kalin, "The chess transformer: Mastering play using generative language models," 2020, *arXiv:2008.04057*.
- [7] D. Monroe and P. A. Chalmers, "Mastering chess with a transformer model," 2024, *arXiv:2409.12272*.
- [8] A. Cook and O. Karakuş, "LLM-commentator: Novel fine-tuning strategies of large language models for automatic commentary generation using football event data," *Knowl.-Based Syst.*, vol. 300, Sep. 2024, Art. no. 112219.
- [9] K. Ge, L. Chen, K. Zhang, Y. Luo, T. Shi, L. Fan, X. Li, G. Wang, and S. Zhang, "SCBench: A sports commentary benchmark for video LLMs," 2024, *arXiv:2412.17637*.
- [10] Z. Wang and N. Yoshinaga, "Commentary generation from multimodal game data for esports moments in multiplayer strategy games," in *Proc. 14th Int. Joint Conf. Natural Lang. Process. 4th Conf. Asia-Pacific Chapter Assoc. Comput. Linguistics*, 2025, pp. 1795–1807.
- [11] A. F. Mohammad, B. Clark, and R. Hegde, "Large language model (LLM) & GPT: a monolithic study in generative AI," in *Proc. Congr. Comput. Sci., Comput. Eng., Appl. Comput. (CSCE)*, Jul. 2023, pp. 383–388.
- [12] E. Almazrouei, H. Alobeidli, A. Alshamsi, A. Cappelli, R. Cojocaru, M. Debbah, É. Goffinet, D. Hesslow, J. Launay, Q. Malartic, D. Mazzotta, B. Noune, B. Pannier, and G. Penedo, "The falcon series of open language models," 2023, *arXiv:2311.16867*.
- [13] M. Aridoss, K. S. Bisht, and A. K. Natarajan, "Comprehensive analysis of falcon 7B: A state-of-the-art generative large language model," in *Generative AI: Current Trends and Applications*. Cham, Switzerland: Springer, 2024, pp. 147–164.
- [14] J. Kim, J. Goh, I. Hwang, J. Cho, and J. Ok, "Bridging the gap between expert and language models: Concept-guided chess commentary generation and evaluation," 2024, *arXiv:2410.20811*.
- [15] M. Soliman and N. Ehab, "Caissa AI: A neuro-symbolic chess agent for explainable move suggestion and grounded commentary," in *Proc. German Conf. Artif. Intell.*, 2025, pp. 148–160.
- [16] B. Oshri and N. Khandwala, (2016). *Predicting Moves in Chess Using Convolutional Neural Networks*. [Online]. Available: <http://vision.stanford.edu/teaching/cs231n/reports/2015/pdfs/ConvChess.pdf>
- [17] S. Toshniwal, S. Wiseman, K. Livescu, and K. Gimpel, "Chess as a testbed for language model state tracking," in *Proc. AAAI Conf. Artif. Intell.*, 2022, vol. 36, no. 10, pp. 11385–11393.
- [18] H. Mellisop, C. Schlager, and C. Swingle, (2021). *Chept-2: Advancing the Application of Deep Neural Transformer Models to Chess Move Prediction and Self-Commentary*. [Online]. Available: <https://cs229.stanford.edu/proj2021spr/report2/81981110.pdf>
- [19] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, T. Lillicrap, and D. Silver, "Mastering atari, go, chess and shogi by planning with a learned model," *Nature*, vol. 588, no. 7839, pp. 604–609, Dec. 2020.
- [20] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis, "A general reinforcement learning algorithm that masters chess, shogi, and go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, Dec. 2018.
- [21] Y. Li, H. Liu, T. B. Pedersen, Y. He, K. G. Larsen, L. Chen, C. S. Jensen, J. Xu, and T. Li, "RobustZero: Enhancing MuZero reinforcement learning robustness to state perturbations," in *Proc. 42nd Int. Conf. Mach. Learn.*, 2025, pp. 4–9.
- [22] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [23] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.
- [24] G. Team et al., "Gemini: A family of highly capable multimodal models," 2023, *arXiv:2312.11805*.
- [25] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.
- [26] P. He, X. Liu, J. Gao, and W. Chen, "DeBERTa: Decoding-enhanced BERT with disentangled attention," in *Proc. Int. Conf. Learn. Represent.*, 2021, pp. 3–9.
- [27] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Natural Lang. Process. 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 3980–3990.
- [28] J. P. Kincaid, R. P. Fishburne Jr., R. L. Rogers, and L. S. Chissom, "Derivation of new readability formulas (automated readability index and flesch reading ease formula) for army enlisted personnel," Naval Technical Training Command Millington TN Research Branch, Inst. Simul. Training, Univ. Central Florida, USA, Tech. Rep. 8-75, 1975.
- [29] P. M. McCarthy and S. Jarvis, "MTLD, VOCD-D, and HD-D: A validation study of sophisticated approaches to lexical diversity assessment," *Behav. Res. Methods*, vol. 42, no. 2, pp. 381–392, May 2010.
- [30] Y. Zhu, S. Lu, L. Zheng, J. Guo, W. Zhang, J. Wang, and Y. Yu, "Txygen: A benchmarking platform for text generation models," in *Proc. 41st Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jun. 2018, pp. 1097–1100.
- [31] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bull.*, vol. 1, no. 6, pp. 80–83, Dec. 1945.
- [32] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Hillsdale, NJ, USA: Lawrence Erlbaum Associates, 1988.

¹ GitHub repository:
<https://github.com/Pranav-Bhatlapenumarthi/KibitzAI>



HITESH SHIVKUMAR is currently pursuing the B.Tech. degree in computer science with Vellore Institute of Technology, Vellore, India. He is extremely passionate and dedicated to building impactful, scalable solutions at all levels. He is actively engaged in multiple research projects along with academic pursuits. His areas of interests include agentic AI, generative AI, LLMs, cloud development, and modern web development.



PRANAV BHATLAPENUMARTHI is currently pursuing the B.Tech. degree in computer science engineering with Vellore Institute of Technology, Vellore, India. He is actively involved in building AI-driven solutions and passionate about interdisciplinary collaborations, aiming to bridge the gap between advanced model capability and real-world human accessibility. His research interests include machine learning, applied deep learning, and generative AI.



SWATHI JAMJALA NARAYANAN (Senior Member, IEEE) received the Ph.D. degree from Vellore Institute of Technology (VIT), Vellore, India, in 2015. She is currently a Professor with the School of Computer Science and Engineering, VIT. With 19 years of teaching experience in computer science, her primary research interests include developing machine learning models for surveillance systems and LLMs for various applications. She is a lifetime member of the Computer Society of India. She has received the Best Ph.D. Thesis Award from the Computer Society of India.

Society of India. She has received the Best Ph.D. Thesis Award from the Computer Society of India.



SABHYA GUPTA is currently pursuing the B.Tech. degree in computer science and engineering with Vellore Institute of Technology, Vellore. He is maintaining a distinguished academic record. As an Oracle-certified generative AI professional, he has actively led the development of impactful solutions in context-aware privacy masking and deployed AI tools for social good. His research and technical expertise span artificial intelligence, NLP, and distributed cloud computing architectures.



TANISHKA SRIVASTAVA is currently pursuing the Bachelor of Technology degree in computer science and engineering with Vellore Institute of Technology, Vellore, India. She is actively involved in a range of academic and technical initiatives, demonstrating a strong commitment to both theoretical learning and practical application. Her primary research interests include machine learning, web development, and programming.



BOOMINATHAN PERUMAL (Senior Member, IEEE) received the Ph.D. degree from Vellore Institute of Technology (VIT), Vellore, India. He is currently a Professor (Grade 1) with the School of Computer Science and Engineering, VIT. He has 21 years of teaching experience in the field of computer science. His research interests include cloud computing, optimization techniques, and machine learning. He is lifetime member of the Computer Society of India. He is also the Faculty

Coordinator of the IEEE Computer Society Student Chapter, VIT.

...